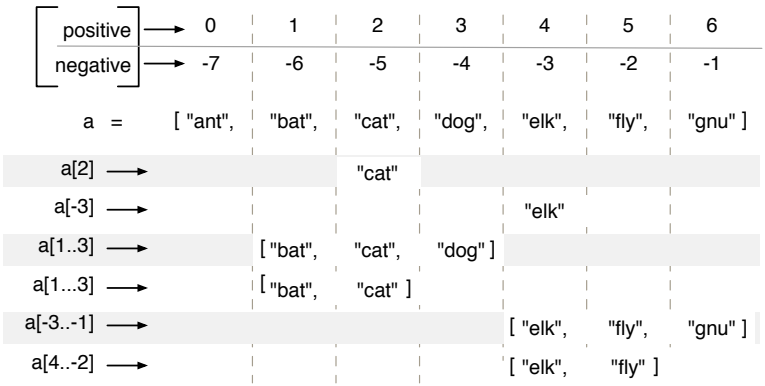
```
a = [1, 3, 5, 7, 9]
a[-1]     # => 9
a[-2]     # => 7
a[-99]    # => nil
```

The following diagram shows array access in a different way:



Arrays are accessed using the [] operator. As with most Ruby operators, this operator is implemented as a method, specifically, an instance method of class Array. The last two lines of this example are equivalent:

```
a = [3.14159, "pie", 99]
a[0]      # => 3.14159
a.[](0)   # => 3.14159
```

The last line of code treats [] as a normal method. In practice, you wouldn't write code like the last line, we just wanted to show how flexible Ruby is.

You can also index arrays with a pair of numbers, [start, count]. This returns a new array consisting of references to count number of objects starting at position start:

```
a = [1, 3, 5, 7, 9]
a[1, 3]   # => [3, 5, 7]
a[3, 1]   # => [7]
a[-3, 2]  # => [5, 7]
```

Finally, you can index arrays using ranges, in which start and end positions are separated by two or three periods. The two-period form includes the end position; the three-period form does not. We'll talk more about ranges in Chapter 7, Standard Types, on page 117.

```
a = [1, 3, 5, 7, 9]
a[1..3]   # => [3, 5, 7]
a[1...3]  # => [3, 5]
a[3..3]   # => [7]
a[-3..-1] # => [5, 7, 9]
```

The [] operator has a corresponding []= operator, which lets you set elements in the array. If used with a single integer index, the element at that position is replaced by whatever is on the right side of the assignment. Any gaps that result will be filled with nil:

```
a = [1, 3, 5, 7, 9]       #=> [1, 3, 5, 7, 9]
```